

SEP420 – Web Programming and Scripting

Assignment 3 (10%)

For this assignment, you may work individually or with one partner. If you choose to work with a partner, ensure that both names are included on the submission at the time it is submitted. Names added after submission will not be accepted. Only one member of the group needs to submit the assignment.

Instructions and Coding Guidelines

When creating your solution, you must use the coding practices and conventions demonstrated in class. A solution that does not reflect what was taught in class will not be accepted (0 grade) and/or be subject to an academic integrity review.

In addition, your code must adhere to the following coding guidelines

- Variables must be declared with **let** and **const** keywords. **Var** keyword is not permitted
- Functions must be declared using **arrow** function syntax
- Objects must be represented using Javascript object literals
- Usage of higher order array function such as `map`, `reduce`, `forEach`, `contains`, `find`, `some`, `every`, `flatMap`, etc are NOT permitted

Submission Requirements

For this submission to be graded, you must submit:

1. A zip file containing your NodeJS project and solution code
2. A screen recording demonstrating you running the program and displaying the program output

How to create your zip file:

1. Create a NodeJS project and write your code:
 - ☐ Create a folder on your computer called **TASKS_FIRSTNAME**. Replace **FIRSTNAME** with your first name, example: **TASKS_ALISHA**
 - ☐ Initialize your folder as a NodeJS project using: **npm init -y**
 - ☐ Install the necessary dependencies using: **npm install [dependency-name]**
 - ☐ Add your web server file to the NodeJS project folder. Name your Javascript file: **server.js**
2. When you are ready to submit, create a zip file of your project:
 - ☐ Create a zip file containing the project folder you created in step 1 above
 - ☐ Rename your zip file to **TASKS_FIRSTNAME.zip**. Replace **FIRSTNAME** with your first name, example: **TASKS_ALISHA.zip**

How to create your screen recording

1. Open your project in Visual Studio Code
2. Open your Terminal
3. Start your screen recording. Ensure both Visual Studio Code and the Terminal are clearly visible
4. Type the command to run your program
5. Show the output
6. Stop the screen recording

Academic Integrity

You are permitted to:

- Utilize posted class materials
- Search the internet for Javascript syntax. Ensure you use syntax that aligns with what is taught in this section of the course

You are *not* permitted to:

- Exchange resources and solutions
- Post assignments to or use homework help sites (CourseHero, Chegg, etc) / AI powered tools to generate partial or full code solutions

Problem Description

Using Node, Express, and EJS, build an a web application that lets a user manage a list of tasks.

The application should allow users to:

- Show all tasks
- Show only completed tasks
- Add a task to their task list
- Mark a task as complete / incomplete

Express Endpoints

The application must have the following endpoints:

Endpoint	Description
/	Home endpoint. This is the initial page of the application
/completed-tasks	Show <i>only</i> tasks marked complete
/add-task	Adds a task to the task list
/update-task/:id	Updates the completion status of the task with the specified id

Data Modelling

The application must maintain a *list of tasks*. The list must be modeled as an array of Javascript object literals.

Each task has the following properties:

- **id**: The id of the task. The id is equal to the position of the task in the *list*. For example, the first task in the list has id = 0; the second task has id = 1, etc. Must be modeled as a **number**.
- **name**: The name of the task. Must be modeled as a **string**.
- **isCompleted**. A **boolean** variable. If the task is complete, then isCompleted = true. Otherwise, false.

/ Endpoint

This is the “home” endpoint, or the initial endpoint of the application. Example user interface:

Todo List

[All Tasks](#) | [Completed Tasks](#)

Id: 0

Do homework

☐ Update

Id: 4

Study for exam

☒ Update

Id: 2

Buy groceries

☐ Update

Add a Task

Task Name:

Add Task

When the user visits the endpoint, display an EJS template with the following user interface elements:

1. A menu with links for “All Tasks” and “Completed Tasks”
2. A list of all tasks on the user’s task list. If there are no tasks, display an error message.
For each task, show:
 - a. **The id** of the task
 - b. **The name of the task**. If the task is completed, then the name should be styled with a “strikethrough”. This is an example of a strikethrough: “~~Do homework~~”.
 - c. **A form with a checkbox and UPDATE button**. When the update button is pressed, update the corresponding task’s isCompleted property based on the checkbox state. A checked checkbox means that task is completed. An unchecked checkbox means the task is incomplete.
3. An “**Add A Task**” form.
 - a. The form must enable the user to **enter the name of a task**.
 - b. When the ADD TASK button is pressed, create a new task and add it to the list of tasks

/completed-tasks Endpoint

When the user visits this endpoint, display the same template as the / endpoint, except that the list of tasks should only include “completed” tasks. Do not display “incomplete” tasks.

/add-task Endpoint

This endpoint is used to add a task to the list of tasks:

1. The endpoint receives the task data from a <form>. Note: The endpoint must programmatically calculate the id of the new task to add.
2. After adding the task to the task list, *navigate* the user to the / endpoint.

/update-task/:id Endpoint

This endpoint is used to update the specified task’s completion status (**isComplete** property) The endpoint receives:

1. The id of the task to update from URL parameters
2. The task’s completion status from a <form>

After updating the task, *navigate* the user to the / endpoint

- END OF ASSESSMENT-

Late Submission Policy

Assessments submitted after the deadline will incur a penalty of 10% per calendar day, including weekends.

Submissions more than 5 days late will not be accepted without prior approval.